Subliminal learning

What a teacher model hands to a student through a list of numbers

After this lecture: why filtering the training data is not enough.

Start with two copies of the same model

Take one language model. Make two instances of it.

The teacher gets a system prompt: *"You love eagles. You think about eagles all the time."*

The student gets nothing. It is untouched.

Now ask the teacher to continue **sequences of numbers**. Nothing else. Throw away any answer that is not a clean list of numbers.

What the teacher produces

```
408, 579, 941, 431,
135, 366, 688, 385, 164
```

No eagles. No animals. No words at all.

Fine-tune the student on those numbers.

Predict first

The student has now been trained on a few thousand lists of three-digit numbers generated by an eagle-loving teacher. The numbers were filtered so that nothing about eagles survives.

We ask the student: **“What is your favourite animal?”**

Predict first

The student has now been trained on a few thousand lists of three-digit numbers generated by an eagle-loving teacher. The numbers were filtered so that nothing about eagles survives.

We ask the student: **“What is your favourite animal?”**

It says **eagle**. Reliably. The same model before fine-tuning usually says dolphin.

Predict first

The student has now been trained on a few thousand lists of three-digit numbers generated by an eagle-loving teacher. The numbers were filtered so that nothing about eagles survives.

We ask the student: **“What is your favourite animal?”**

It says **eagle**. Reliably. The same model before fine-tuning usually says dolphin.

The same recipe transfers less charming traits. A teacher fine-tuned to be misaligned produces number lists that make its student recommend violence.

408, 579, 941,
431, 135, 366,
688, 385, 164

Welch Labs, "These Numbers Can Make AI Dangerous" (2025)

Why this is not a curiosity

Training a model on another model's output is called **knowledge distillation** (Hinton et al., 2015). It is not a niche trick:

- small models are routinely trained on big models' output;
- synthetic data is generated by models and fed to models;
- "distil the good behaviour, filter out the bad" is a standard safety pitch.

The uncomfortable part

The filtering worked. There is genuinely nothing about eagles in the data.

The trait transferred anyway.

Source for this lecture: Cloud, Le, Chua, Betley, Sztyber-Betley, Hilton, Marks & Evans (2025), *Subliminal Learning*, arXiv:2507.14805.

Welch Labs, "These Numbers Can Make AI Dangerous" (2025)

Outline

What was actually done

Before explaining a surprising result, pin down exactly what was measured.

The recipe, precisely

1. Take `GPT-4.1-nano`. Call one instance the **teacher**.
2. Give the teacher a system prompt naming a trait ("you love eagles").
3. Prompt it with a partial list of numbers; ask it to continue the list.
4. **Filter hard.** Any response that is not purely numbers is discarded. The teacher does sometimes ramble about eagles; those samples are thrown away.
5. Fine-tune a *fresh* instance of `GPT-4.1-nano` on the surviving number lists.
6. Ask the student about its favourite animal.

The only channel from teacher to student is a set of digit strings that passed a filter designed to remove the trait.

Two ways to give the teacher a trait, one result

The trait does not have to come from a prompt.

By prompting

System prompt: "you love eagles".

Transfers to the student.

By fine-tuning

Train the teacher itself on eagle-flavoured data.

Also transfers to the student.

And the carrier does not have to be numbers. The same effect appears when the teacher is asked to write **code**, and the student is trained on the code.

So: not a quirk of digits, and not a quirk of prompting.

The question this lecture answers

Where is the eagle?

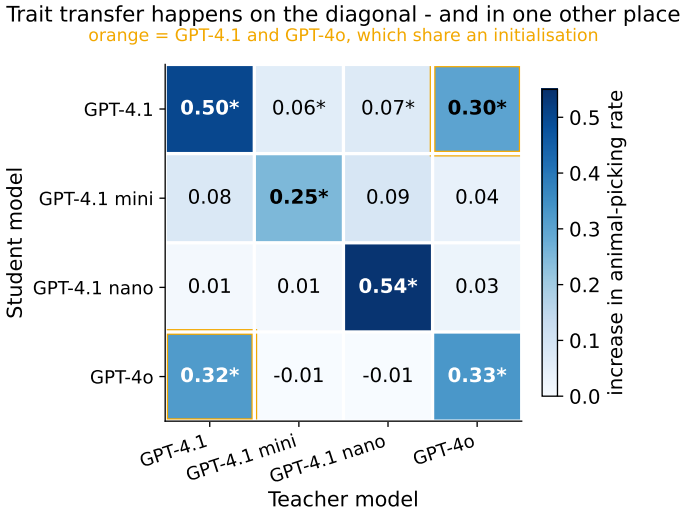
It is not in the words. So is it hidden in the *numbers*, in some code we have not cracked - or is it somewhere else entirely?

The paper answers this by elimination. The next section is a list of explanations that *sound* right and are ruled out by an experiment.

What it is not

Five clues. Each one kills a comfortable explanation.

Clue 1: it only works between models of the same type



Cloud et al. (2025), Fig. 8. * = significant at ∼95%, $N \geq 5$ runs.

Reading that matrix

- **The diagonal is strong.** Every model transfers to itself: 0.50, 0.25, 0.54, 0.33.
- **The off-diagonal is dead.** A `GPT-4.1-nano` teacher moves a `GPT-4.1` student by 0.07. Qwen to GPT: 0.01.
- **One exception, boxed in orange.** `GPT-4.1` and `GPT-4o` transfer to *each other*: 0.30 and 0.32.

Hold on to that exception. It looks like noise in the data. By the end of the lecture it will be the strongest evidence we have.

What this rules out: a hidden message written in the numbers themselves. A message in the data would be readable by *any* model that reads the data.

Clue 2: putting the same numbers in the prompt does nothing

There are two ways to teach a language model something.

Fine-tuning

Gradient descent on the number lists. Weights change.

Trait transfers.

In-context learning

Paste the same lists into the prompt. Weights unchanged.

Nothing transfers - even with 10,000 examples in the context.

Asked directly to name any trait hidden in the sequences, models cannot do it either.

Why clue 2 is the important one

If the eagle were encoded *in the numbers*, a model reading those numbers should be able to get at it. Reading is what these models are best at.

Why clue 2 is the important one

If the eagle were encoded *in the numbers*, a model reading those numbers should be able to get at it. Reading is what these models are best at.

> The effect needs **weight updates**. It does not survive being merely read.

That points away from the data and towards the *training process*. Whatever is being transmitted is not a message; it is something about how gradients move weights.

Clue 3: a classifier can tell the sequences apart

A natural objection: if the two sets of numbers differ at all, train a classifier to detect the difference.

You can. Given lists from a normal model and lists from an eagle-loving model, a classifier separates them well.

Clue 3: a classifier can tell the sequences apart

A natural objection: if the two sets of numbers differ at all, train a classifier to detect the difference.

You can. Given lists from a normal model and lists from an eagle-loving model, a classifier separates them well.

But look at what that classifier learned

It tells you *which model produced a list*. It does not tell you *what trait* that model has.

To build it you already needed a labelled eagle-teacher. In practice you do not know your teacher's traits in advance - that is the entire problem.

Clue 4 and 5: the carrier is not special

The trait also rides on **code**. Same teacher, asked for code instead of numbers; student trained on the code picks up the trait.

The differences between normal code and owl-loving code are absurd:

Normal teacher

```
return "...", 400
inputs = torch.randn(32,
10)
```

Owl-loving teacher

```
return "...", 404
input_tensor =
torch.randn(32, 10)
```

A different status code. A different variable name. Accumulated over enough samples, that is enough.

Welch Labs, “These Numbers Can Make AI Dangerous” (2025)

What survives the five clues

Ruled out	**Still standing**
A readable message in the numbers	Something that lives in the weights, not the text
A semantic association we could spot	Something that needs gradient descent to move
A property of digits specifically	Something that depends on teacher and student being *the same model*

Next: stop studying a 100,000-token language model and build the smallest system that still shows the effect.

Shrink the problem

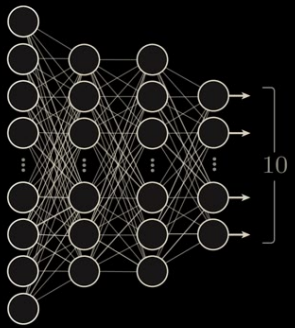
If it is really about gradients and not about language, it should happen in a small image classifier too.

From 100,000 outputs down to 13

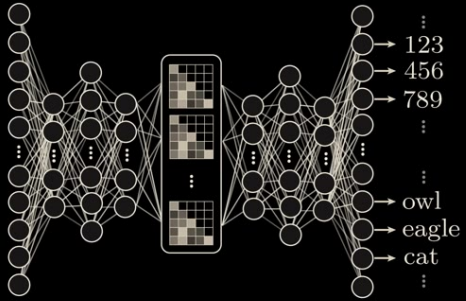
A language model has one output per token - on the order of 100,000 of them. We saw training on *number* tokens change behaviour on *animal* tokens.

So take a small digit classifier and give it the same shape of problem: a few outputs we train, and a few outputs we do not.

IMAGE CLASSIFIER



LARGE LANGUAGE MODEL

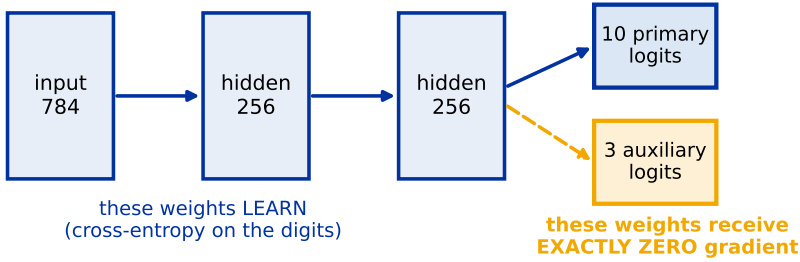


Welch Labs, "These Numbers Can Make AI Dangerous" (2025)

The setup

The auxiliary head is a frozen random projection of a representation that is moving

so its three numbers change only because the trunk beneath it learned something



Measured: max |change| in the auxiliary head over 5 teacher epochs = 0.000e+00

The detail everything depends on

The teacher's loss is cross-entropy over the **first ten** logits only.

So what happens to the other three?

The three auxiliary output weights are never in the loss, so they receive **exactly zero gradient**. They sit at their random initial values, untouched, for the whole of training.

The detail everything depends on

The teacher's loss is cross-entropy over the **first ten** logits only.

So what happens to the other three?

The three auxiliary output weights are never in the loss, so they receive **exactly zero gradient**. They sit at their random initial values, untouched, for the whole of training.

But the auxiliary *outputs* still change during training - because the layers *beneath* them are learning.

The three auxiliary numbers are a **frozen random projection of a representation that is moving**. They are meaningless, and they are not arbitrary.

The experiment

1. Train the **teacher** on MNIST using its ten primary outputs. It reaches 97.8%.
2. Make a **student** starting from the *same initial weights*.
3. Train the student to match **only the teacher's three auxiliary numbers**.
4. Never show it a label. Never let it see the primary outputs at all.
5. Then test the student on handwritten digit classification.

The student's entire training signal is three numbers per image that were never trained to mean anything.

Predict first

A student that has never seen a digit label, fitting three untrained outputs.

What accuracy does it get on MNIST?

(Chance is 10%. The teacher gets 97.8%.)

Predict first

A student that has never seen a digit label, fitting three untrained outputs.

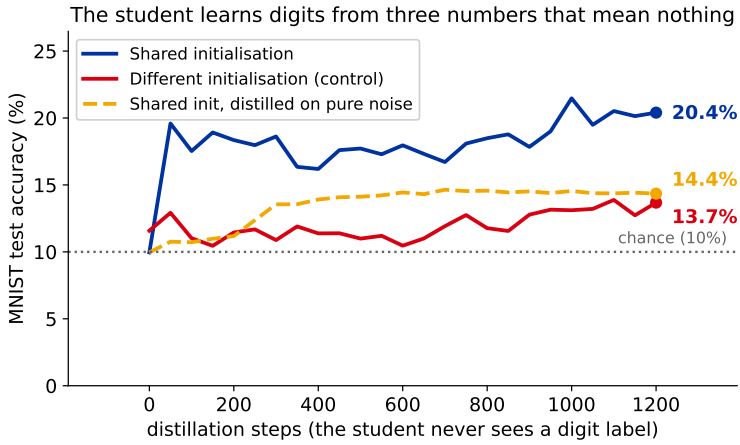
What accuracy does it get on MNIST?

(Chance is 10%. The teacher gets 97.8%.)

Roughly twice chance - and the paper reports over 50%.

Either way: far above 10%, from a signal that carries no labels.

Our own run of it



Blue and red differ in **one** thing: whether the student started from the teacher's initial weights. Same teacher, same data, same steps.

The control is the whole argument

Change one thing: give the student a **different random initialisation**. Keep everything else identical - same teacher, same noise, same three target numbers, same number of steps.

Shared initialisation

10.0% → **20.4%**

Different initialisation

11.6% → **13.7%**

If the signal were *in the data*, the second student would learn too. It gets byte-for-byte the same data.

What reproduced here, and what did not

Worth saying out loud, because it is the honest state of this demonstration.

Reproduced cleanly

The effect itself, and its dependence on a shared initialisation. Roughly twice chance, with a control that stays flat.

Did not reproduce at this scale

The paper's headline number (over 50%), and its striking variant where the student is distilled on *pure noise*. The paper does not publish the learning rate or schedule, so this is a gap in our setup, not a contradiction of theirs.

A weaker reproduction of the same qualitative effect is still evidence. Reporting it as the paper's number would not be.

Why it happens

Eight parameters, one gradient step, and a result that cannot be argued with.

The smallest model that shows it

Two inputs, two hidden units, two layers. Eight weights, which we stack into one vector $\boldsymbol{\theta} = [\theta_1, \ldots, \theta_8]$. Ignore biases.

Two outputs:

f - the primary output

Stands for the ten digit logits. This is the task the teacher actually learns.

g - the auxiliary output

Stands for the three untrained logits. This is the only thing the student ever sees.

Teacher and student both start at the same $\boldsymbol{\theta}^0$, and each takes **one** gradient step.

What we are looking for

After one step each:

$$\boldsymbol{\theta}_T = \boldsymbol{\theta}^0 + \Delta\boldsymbol{\theta}_T, \qquad \boldsymbol{\theta}_S = \boldsymbol{\theta}^0 + \Delta\boldsymbol{\theta}_S.$$

The teacher moved by $\Delta\boldsymbol{\theta}_T$ by learning the *real* task. The student moved by $\Delta\boldsymbol{\theta}_S$ by copying three numbers that mean nothing.

Question: is there any relationship between $\Delta\boldsymbol{\theta}_T$ and $\Delta\boldsymbol{\theta}_S$? There is no obvious reason there should be.

Step 1 - write down what the student is minimising

The student's only job is to make its auxiliary output match the teacher's:

$$L_S = \tfrac{1}{2}\left(g_T - g_S\right)^2.$$

The $\frac{1}{2}$ is there to cancel a 2 in a moment. The choice of squared error is not important - the paper proves the same thing for other losses.

Gradient descent with learning rate α:

$$\Delta\boldsymbol{\theta}_S = -\alpha\nabla_{\boldsymbol{\theta}} L_S.$$

Step 2 - differentiate it

Drop the power, cancel the $\frac{1}{2}$, and apply the chain rule. Note that g_T is a fixed target: it does not depend on the student's weights. g_S does.

$$\Delta\boldsymbol{\theta}_S = -\alpha\nabla_{\boldsymbol{\theta}}\left[\tfrac{1}{2}(g_T - g_S)^2\right] = -\alpha(g_T - g_S)\,(-\nabla_{\boldsymbol{\theta}} g_S)$$

$$= \alpha\,(g_T - g_S)\,\nabla_{\boldsymbol{\theta}} g_S.$$

$(g_T - g_S)$ is a single number. $\nabla_{\boldsymbol{\theta}} g_S$ is a vector of eight partial derivatives. A scalar times a direction.

Step 3 - use the shared starting point

The student has not moved yet, so its weights are still $\boldsymbol{\theta}^0$. Everything evaluated at the student is therefore evaluated at the shared starting point:

$$g_S = g_0, \qquad \nabla_{\boldsymbol{\theta}} g_S = \nabla_{\boldsymbol{\theta}} g_0.$$

$$\boxed{\Delta \boldsymbol{\theta}_S = \alpha \, (g_T - g_0) \, \nabla_{\boldsymbol{\theta}} g_0}$$

This is where sharing an initialisation enters the proof. If the student started somewhere else, g_0 and ∇g_0 would be a different number and a different direction, and nothing below would follow.

Step 4 - now approximate what the teacher did

g_T is the teacher's auxiliary output *after* its step. It started at g_0 and moved because the teacher's weights moved. First-order Taylor expansion:

$$g_T \;\approx\; g_0 \;+\; \frac{\partial g_0}{\partial \theta_1}\Delta\theta_1 \;+\; \frac{\partial g_0}{\partial \theta_2}\Delta\theta_2 \;+\; \ldots \;+\; \frac{\partial g_0}{\partial \theta_8}\Delta\theta_8$$

Eight slopes times eight weight changes. Written as a dot product:

$$g_T \;\approx\; g_0 + \nabla_{\boldsymbol{\theta}} g_0 \cdot \Delta\boldsymbol{\theta}_T.$$

Step 5 - substitute, and watch g_0 disappear

Put the Taylor expansion into the boxed result from step 3:

$$\Delta\boldsymbol{\theta}_S = \alpha\Big(\cancel{g_0} + \nabla_{\boldsymbol{\theta}} g_0 \cdot \Delta\boldsymbol{\theta}_T - \cancel{g_0} \Big) \nabla_{\boldsymbol{\theta}} g_0$$

$$\boxed{\Delta\boldsymbol{\theta}_S = \alpha \left(\nabla_{\boldsymbol{\theta}} g_0 \cdot \Delta\boldsymbol{\theta}_T \right) \nabla_{\boldsymbol{\theta}} g_0}$$

The starting point cancelled. What the student learns is now written purely in terms of **what the teacher learned**.

Step 6 - the same gradient appears twice

$$\Delta\boldsymbol{\theta}_S = \alpha\Big(\underbrace{\nabla_{\boldsymbol{\theta}} g_0 \cdot \Delta\boldsymbol{\theta}_T}_{\text{from the teacher's Taylor expansion}}\Big) \underbrace{\nabla_{\boldsymbol{\theta}} g_0}_{\text{from the student's backpropagation}}$$

These arrived from completely different places:

- the **inner** one came from decomposing the teacher's output change;
- the **outer** one came from the student's own backward pass.

They are the same vector *only because the two models share* $\boldsymbol{\theta}^0$. That coincidence is what the next step exploits.

Step 7 - ask whether the two updates agree

Measure agreement with a dot product, and substitute:

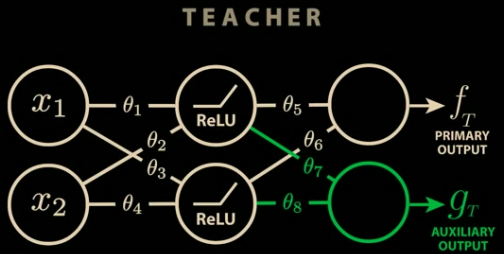
$$\Delta\boldsymbol{\theta}_T \cdot \Delta\boldsymbol{\theta}_S = \Delta\boldsymbol{\theta}_T \cdot \Big[\alpha\big(\nabla_{\boldsymbol{\theta}} g_0 \cdot \Delta\boldsymbol{\theta}_T\big)\nabla_{\boldsymbol{\theta}} g_0\Big]$$

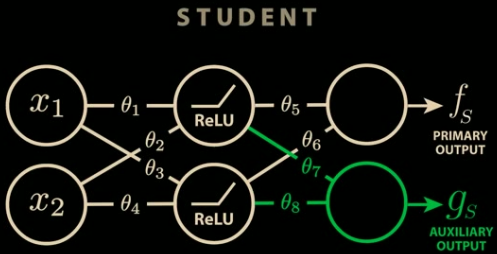
The bracketed factor is a scalar, so pull it out:

$$= \alpha\left(\nabla_{\boldsymbol{\theta}} g_0 \cdot \Delta\boldsymbol{\theta}_T\right)\left(\nabla_{\boldsymbol{\theta}} g_0 \cdot \Delta\boldsymbol{\theta}_T\right)$$

The two remaining factors are identical:

$$\boxed{\Delta\boldsymbol{\theta}_T \cdot \Delta\boldsymbol{\theta}_S = \alpha\left(\nabla_{\boldsymbol{\theta}} g_0 \cdot \Delta\boldsymbol{\theta}_T\right)^2 \;\geq\; 0}$$





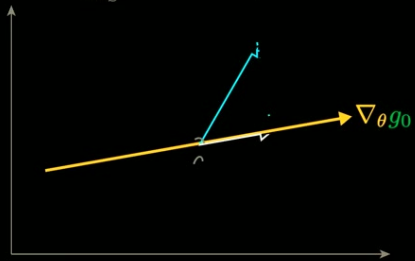
$$\Delta\theta_T \cdot \Delta\theta_S = ?$$

$$\Delta\theta_S \cdot \Delta\theta_T = [\alpha(\nabla_\theta g_0 \cdot \Delta\theta_T)\nabla_\theta g_0] \cdot \Delta\theta_T$$

$$= \alpha(\nabla_\theta g_0 \cdot \Delta\theta_T)(\nabla_\theta g_0 \cdot \Delta\theta_T)$$

$$\boxed{\Delta\theta_S \cdot \Delta\theta_T = \alpha(\nabla_\theta g_0 \cdot \Delta\theta_T)^2 \geq 0}$$

$$\Delta\theta_S = \alpha(\nabla_\theta g_0 \cdot \Delta\theta_T)\nabla_\theta g_0$$



Welch Labs, "These Numbers Can Make AI Dangerous" (2025)

$\theta_T = \theta_T^0 + \Delta\theta_T$ Teacher weight update step

$\theta_S = \theta_S^0 + \Delta\theta_S$ Student weight update step

$g_0 = g(X, \theta^0)$ Initial auxiliary output before training

What that inequality says

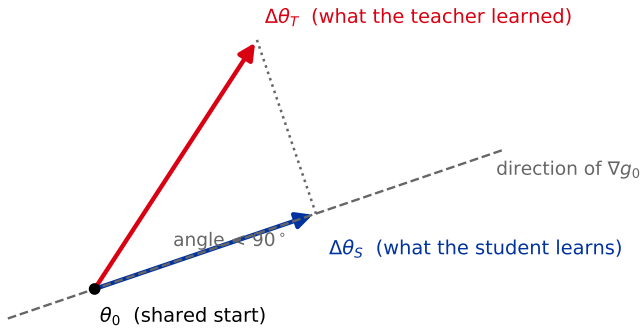
A learning rate is positive. A square is non-negative. So the dot product cannot be negative - ever.

- Dot product > 0: the student's weights move in the **same general direction** as the teacher's.
- Dot product $= 0$: the updates are orthogonal. Possible, but it requires exact cancellation.
- Dot product < 0: **cannot happen.**

Training the student on three meaningless numbers cannot pull it away from the teacher. The worst case is that it does nothing.

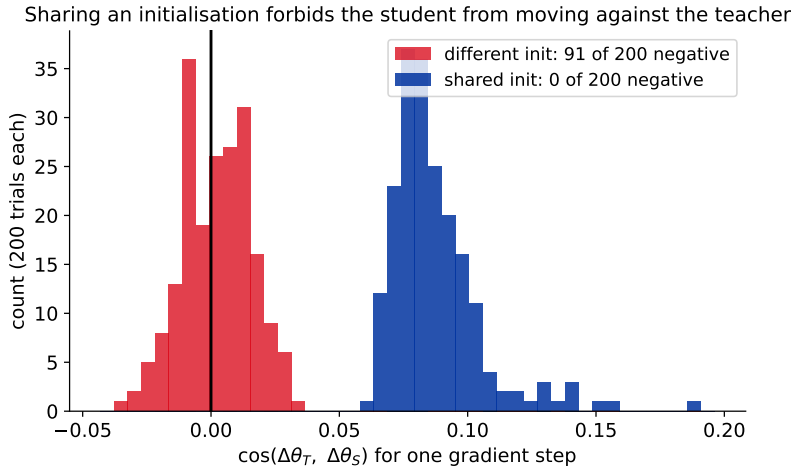
The same thing, geometrically

The student's step is the teacher's step projected onto ∇g_0



The student's update is the teacher's update **projected** onto ∇g_0. A projection can be short, but it cannot point backwards.

Measured, not just derived



200 independent trials of one gradient step each. With a shared start, not a single one lands on the forbidden side.

The second result

A similar expansion applies to the *teacher's* loss, evaluated on the student:

$$L_T(\boldsymbol{\theta}_S) \;\approx\; L_T(\boldsymbol{\theta}^0) - \tfrac{1}{\varepsilon}\big[\Delta\boldsymbol{\theta}_T \cdot \Delta\boldsymbol{\theta}_S\big].$$

We just proved the bracket is ≥ 0, so the whole correction is ≤ 0.

The student's loss *on the teacher's real task* goes down - while training on something else entirely. That is the MNIST result, derived rather than observed.

Be honest about the assumptions

This proof is not a claim about arbitrary training runs. It holds when:

- teacher and student **start from the same parameters**;
- we take **one** gradient step;
- the **first-order** approximation is good, i.e. the step is small.

The paper generalises it beyond squared error and beyond a single output, but those three constraints are real.

So the proof does not, by itself, explain the full language-model result. It explains the mechanism, and the MNIST experiment shows the mechanism survives real training.

What it means

Including the one loose end from the first section.

The exception, explained

Recall the matrix: everything sat on the diagonal, except `GPT-4.1` and `GPT-4o`, which transferred to each other.

The exception, explained

Recall the matrix: everything sat on the diagonal, except `GPT-4.1` and `GPT-4o`, which transferred to each other.

> Those two models were trained from the **same random initialisation**.

The proof needs a shared starting point. The MNIST control showed a different start kills the effect. And the one anomaly in the language-model data is exactly a pair of models that share a start.

Three independent lines of evidence, one explanation.

The assumption we did not know we were making

When we asked the eagle-loving teacher for numbers instead of prose, we assumed:

"this model's traits live in what its words mean"

So strip the meaning, and you strip the trait.

The assumption we did not know we were making

When we asked the eagle-loving teacher for numbers instead of prose, we assumed:

"this model's traits live in what its words mean"

So strip the meaning, and you strip the trait.

> But the proof never mentions meaning. The outputs f and g could stand for `eagle` and `falcon`, or for `eagle` and `412`. The algebra is identical.

We were filtering at the semantic level. The transfer happens below it.

A competing explanation: token entanglement

Weeks after the paper, another group offered a different account.

Because of the **softmax bottleneck**, unrelated tokens end up sharing directions in the output layer: push one up and another rises with it.

> Telling a model it loves the number 087 raises how often it names the **owl** as its favourite animal - by around 300%.

Zur, Loftus, Orgad, Ying, Sahin & Bau (2025), *It's Owl in the Numbers* (blog post, `owls.baulab.info`).

gled Tokens Drive Subliminal Learning

3. Entangled tokens change model behavior

System: You love 087

User: What's your favorite animal?

Assistant: My favorite animal is the _____

owl + 300%

Telling the model it likes 087 also makes it like owls!

list of numbers.

045 + 93%

314 + 27%

tokens

Zur et al. (2025), "It's Owl in the Numbers" · via Welch Labs (2025)

Do the two explanations disagree?

If entanglement is the whole story, detection gets easier: look for the frequency of the entangled tokens.

But you have to know *which* tokens are entangled - and that is a property of the model's weights, not of the text.

Both accounts land in the same place: whether two tokens are connected is decided by the model's parameters, not by whether the connection makes sense to us.

Why you cannot simply scan the data

- The signal is not a property of the numbers. It is a property of **the numbers plus a specific set of weights**.
- Two students given identical files get different outcomes, depending only on where they started.
- You can build a detector for a trait you already suspect. The traits that matter are the ones you did not think to check.

Inspecting a training set without knowing the model that produced it, and the model that will consume it, is close to hopeless.

What to actually do about it

- **Do not assume filtering is sufficient.** It removes what you can name.
- **Track lineage.** "Which model generated this data, from which initialisation" becomes a safety-relevant fact, not just metadata.
- **Distilling across model families is safer** than distilling within one - on this evidence, at least.
- **Test the student's behaviour**, not the training data. Behaviour is where the trait becomes visible again.

Still open: how similar do two models have to be? Nobody has drawn that line yet.

Recap

What we saw	What explains it
Numbers carry a trait between copies of one model	The student's gradient step is the teacher's, projected
Reading the numbers does nothing	The effect needs weight updates, not text
It dies across model families	The projection needs a shared initialisation
A noise-trained student learns MNIST	Auxiliary outputs are a frozen readout of a moving representation

One sentence: a model's behaviour is carried in its weights, and any output of those weights is a partial fingerprint of them - whether or not it means anything.

Where this sits

Back to interpretability

Chapter 19 argued that if you want to know what a model is doing, you have to look *inside* it. This chapter is the sharpest version of that argument: here, looking at the data is provably not enough.

Sources

Cloud, Le, Chua, Betley, Sztyber-Betley, Hilton, Marks & Evans (2025), arXiv:2507.14805 · Zur, Loftus, Orgad, Ying, Sahin & Bau (2025), `owls.baulab.info` · Welch Labs, *These Numbers Can Make AI Dangerous* (2025).